

Project: Multi-Protocol Traffic Generation and Analysis

1.1 Objective

Design, implement, and deploy a containerized distributed system that generates diverse, concurrent network traffic across multiple transport and application protocols. The system must produce configurable traffic patterns using TCP and UDP streams alongside application-level protocols (HTTP/2, QUIC, MQTT) — all running simultaneously from orchestrated containers on one or more lab machines.

The project integrates all course topics: container orchestration (Docker Compose), protocol implementation (HTTP/2, QUIC/HTTP/3, MQTT, raw TCP/UDP), traffic analysis (Wireshark), resilience patterns, and distributed system operation.

1.2 System Requirements

System Architecture

The system must consist of the following containerized components, all orchestrated via Docker Compose:

1. **Traffic Controller** (1 container): Central configuration service that reads a YAML configuration file and coordinates all generators. Exposes a REST API (HTTP/2 or HTTP/1.1) for starting/stopping traffic generation, changing configuration at runtime, querying current status and statistics. Must log all configuration changes with timestamps.
2. **HTTP/2 Traffic Generator** (1 container): Sends configurable HTTP/2 requests (GET, POST with variable payload sizes) to HTTP/2 target servers. Configurable parameters: request rate (requests/sec), payload size (bytes), request method distribution, target endpoints. Must support multiplexed streams (multiple concurrent requests over a single connection).
3. **QUIC/HTTP/3 Traffic Generator** (1 container): Sends configurable QUIC/HTTP/3 requests to a QUIC-capable target server. Configurable parameters: request rate, payload size, stream count, 0-RTT usage. Use aioquic or a similar Python QUIC library.
4. **MQTT Traffic Generator** (1 container): Publishes and subscribes on configurable MQTT topics with configurable QoS levels (0, 1, 2). Configurable parameters: publish rate (messages/sec), payload size, topic count, QoS level distribution. Connects to a Mosquitto broker (separate container).
5. **TCP/UDP Raw Traffic Generator** (1 container): Sends raw TCP and UDP streams with configurable parameters: packet rate, packet size, TCP vs. UDP ratio, burst patterns (constant, periodic burst, random).

6. **Target Services** (2-3 containers): HTTP/2 server (e.g., Hypercorn + FastAPI with HTTP/2), QUIC server (aioquic-based), MQTT broker (Mosquitto). These receive and acknowledge traffic from the generators.
7. **Metrics Collector** (1 container): Aggregates statistics from all generators: packets sent per protocol, bytes transferred, error rates, active connections. Exposes a `/metrics` endpoint (JSON). Optionally provides a simple web dashboard.

All components communicate on a shared Docker network. The entire system must be startable with a single `docker-compose up` command.

1.3 Configuration

Traffic Configuration

The system reads a central YAML configuration file that defines the traffic profile. This file must support:

- Per-protocol parameters (rate, payload size, pattern)
- Sending patterns: constant rate, periodic bursts (configurable burst duration and interval), ramping (linear increase over time), random (Poisson-distributed intervals)
- Global parameters: total duration, warm-up period, cool-down period
- Multiple “phases” that change the traffic mix over time (e.g., phase 1: mostly HTTP/2, phase 2: mostly MQTT, phase 3: all protocols equally)

A sample configuration:

```
1  global:
2    duration: 300
3    warmup: 30
4    cooldown: 30
5
6  phases:
7    - name: http2_dominant
8      duration: 100
9      protocols:
10        http2: {rate: 100, payload_size: 4096}
11        mqtt: {rate: 10, qos: 1}
12        tcp: {rate: 5}
13    - name: balanced
14      duration: 100
15      protocols:
16        http2: {rate: 50}
17        quic: {rate: 50}
18        mqtt: {rate: 30}
```

1.4 Analysis Requirements

Traffic Analysis

Students must perform Wireshark captures and analysis for the following tasks:

1. **Protocol Distribution:** Capture at least 60 seconds of traffic with all generators active. Export protocol hierarchy statistics (Wireshark Statistics → Protocol Hierarchy). Show the distribution of TCP vs. UDP, and within TCP the breakdown of HTTP/2, QUIC, MQTT.
2. **Temporal Analysis:** Use Wireshark I/O graphs to show traffic volume over time for each protocol. Demonstrate that configured sending patterns (constant, burst, ramp) are visible in the captures.
3. **Behavioral Fingerprinting:** Analyze traffic characteristics: packet size distributions per protocol, inter-arrival times, connection patterns. Document which protocol characteristics are distinguishable and which overlap.
4. **Failure Visibility:** Deliberately stop one generator or one target service while capturing. Document how the failure manifests: which protocol-specific indicators appear (TCP RST, MQTT disconnect, QUIC connection close frames)? How quickly is failure visible?
5. **Multi-System Deployment:** Deploy your system across at least 2 lab machines (generators on one, targets on another). Capture traffic on the inter-machine link. Compare with single-machine captures.

1.5 Deliverables

1. **Complete source code:** All Dockerfiles, `docker-compose.yml`, application code, configuration files. Must build and run with `docker-compose build && docker-compose up`.
2. **Traffic configuration profiles:** At least 3 different YAML files demonstrating different traffic mixes (e.g., HTTP/2-heavy, MQTT-heavy, balanced).
3. **Wireshark captures:** At least 5 `.pcapng` files, one per analysis task listed above.
4. **Technical report** (8-12 pages, IEEE format):
 - System architecture with diagram
 - Implementation decisions and trade-offs
 - Configuration format documentation
 - Traffic analysis results with Wireshark screenshots and statistics
 - Failure analysis results
 - Single-machine vs. multi-machine comparison
 - Reflection on protocol behavior under different traffic patterns
 - AI usage documentation (which tools, for what purpose, how verified)
5. **Live demonstration** (15-20 minutes): Start the system, show configuration changes at runtime, display Wireshark captures, demonstrate failure scenarios.

1.6 Grading Criteria

Criterion	Weight	Description
System completeness and functionality	25%	All components implemented and running, clean Docker Compose orchestration, single-command startup
Configuration flexibility	15%	Multiple traffic patterns, runtime re-configuration, phase-based profiles
Traffic analysis depth	25%	Quality of Wireshark analysis, protocol distribution, temporal patterns, behavioral observations, failure visibility
Multi-system deployment	10%	Successful deployment across 2+ machines, inter-machine traffic capture and comparison
Report quality	15%	Structure, clarity, analysis depth, proper citations, reproducibility
Live demonstration	10%	System runs live, configuration changes demonstrated, clear explanation of observations

Grading Philosophy

The more aspects of the system you implement well, configure flexibly, analyze thoroughly, and present clearly, the better your grade. Partial implementations are accepted — a system with fewer protocols but excellent analysis and documentation scores higher than a complete system with superficial treatment.

1.7 Resources

- aioquic (Python QUIC): <https://github.com/aioquic/aioquic>
- Hypercorn (HTTP/2 ASGI server): <https://github.com/pgjones/hypercorn>
- Eclipse Mosquitto: <https://mosquitto.org/>
- paho-mqtt (Python MQTT): <https://github.com/eclipse/paho.mqtt.python>
- Scapy (raw packet crafting): <https://scapy.net/>
- Docker Compose documentation: <https://docs.docker.com/compose/>